Local search with the list of improving moves

Paweł Charkiewicz 160288, Piotr Franc 160306

Objective = cycle length (Euclidean, rounded to nearest integer) + sum of selected node costs. Lower is better.

Pseudocode

All methods read the instance into masterArray, which stores every node (x, y, cost). M = number of runs = 200, N = solution size = ⌈n/2⌉, d(a, b) = Euclidean distance rounded to the nearest integer, s = the current solution (a cycle s[0] → s[1] → ... → s[N − 1] → s[0]), prev(v) / next(v) = the nodes before / after node v on the cycle, Δ = change of the objective (negative = improvement). Both methods use the neighbourhood that was the best in assignment 3: the exchange of a selected and a not selected node (inter-route) + the two-edges exchange (intra-route), steepest search, random starting solutions. Both are run on the same 200 starting solutions. All Δ are computed from the changed edges only.

Moves

```
Node exchange (a, v, b, u): removes edges (a, v), (v, b), puts the not selected node u in place of v.
    Δ = d(a, u) + d(u, b) + cost(u) − d(a, v) − d(v, b) − cost(v)
    Applicable: u is not selected and v lies between a and b (the direction does not matter).
Two-edges exchange (a, a', b, b'): removes edges a → a' and b → b', adds edges (a, b) and (a', b').
    Δ = d(a, b) + d(a', b') − d(a, a') − d(b, b')
    Same relative direction: a → a' and b → b' in s, or a' → a and b' → b (both reversed).
    Apply: reverse the path a' … b (both reversed: the path a … b'); the shorter side of the
    cycle is reversed, which gives the same cycle.
    Relatively inverted move of the edges a → a', b → b' of s: (a', a, b, b'), adds (a', b), (a, b').
    It is not applicable now, but it is after a later move reverses exactly one of the two edges.
    One move has 4 notations: (a, a', b, b'), (a', a, b', b), (b, b', a, a'), (b', b, a', a);
    LM keeps the one starting with the smallest node, so it holds no duplicates.
```

Baseline: steepest local search

```
1. s = random starting solution.
2. Repeat:
3.     Evaluate every node exchange (selected v, not selected u) and every two-edges exchange
       of two non-adjacent edges of s.
4.     If the best Δ ≥ 0: s is a local optimum, stop.
5.     Apply the best move.
```

LM: steepest local search with the list of improving moves

```
1. s = random starting solution, LM = empty list of moves ordered by Δ (best first).
2. Evaluate all moves of s as in the baseline, plus for every pair of edges the relatively
   inverted two-edges exchange; add every move with Δ < 0 to LM.
3. Repeat:
4.     Browse LM from the best move:
         a removed edge is not in s any more (or u is selected)  -> remove the move from LM
         removed edges in s, different relative direction        -> keep it, browse further
         removed edges in s, same relative direction (or both
         reversed)                                               -> this is the move m, stop
5.     If m was found: remove m from LM, apply it, add the new improving moves (next block).
6.     Else if LM was just filled in step 2 or 7: s is a local optimum, stop.
7.     Else evaluate all moves of s as in step 2 (a safety net: finds nothing new in practice).
```

LM: new moves after applying a move

```
1. changed = the nodes whose neighbours changed:
       node exchange (a, v, b, u): a, u, b     two-edges exchange (a, a', b, b'): a, a', b, b'
2. For every changed node w and every not selected node u: evaluate the node exchange of w, u.
3. After a node exchange v is not selected any more: evaluate putting v in place of every
   selected node.
4. Every new edge is w → next(w) for a changed node w. For every such edge and every other edge
   of s sharing no node with it: evaluate the two-edges exchange and its inverted version.
5. Add every evaluated move with Δ < 0 to LM. All other moves keep the Δ evaluated earlier:
   Δ depends only on the removed and added edges, which did not change.
```

RANDOM: random starting solution

```
1. Shuffle all node indices (one generator with seed 42 for all 200 starts).
2. The first N of them, in that order, form the cycle.
```

Computational experiment

Instances: TSPA, TSPB. Two methods (baseline steepest LS and steepest LS with the list of improving moves), each run 200 times from the same random starting solutions. Each cell: average (min – max) over the valid runs; the best average on each instance is in bold.

Objective function

Method	TSPA	TSPB
Steepest LS, baseline (all moves evaluated every iteration)	**76182.18 (73856 – 79249)**	52591.14 (50402 – 56117)
Steepest LS, list of improving moves (LM)	76187.86 (73856 – 79249)	**52572.55 (50402 – 56129)**

Running time [µs]

Time of one run: the local search from the random starting solution (for LM including the initial evaluation of all moves).

Method	TSPA	TSPB
Steepest LS, baseline (all moves evaluated every iteration)	289143.59 (241617 – 437901)	284199.92 (242443 – 338103)
Steepest LS, list of improving moves (LM)	**26406.19 (23072 – 50000)**	**25406.90 (22560 – 28628)**

Average speed-up of LM over the baseline (average baseline time / average LM time): TSPA 10.95×, TSPB 11.19×.

Validation

Valid = the run has ⌈n/2⌉ distinct nodes of the instance. Local optima = no improving move remains in the method's neighbourhood (checked independently by the report script; "best Δ" is the most improving move left).

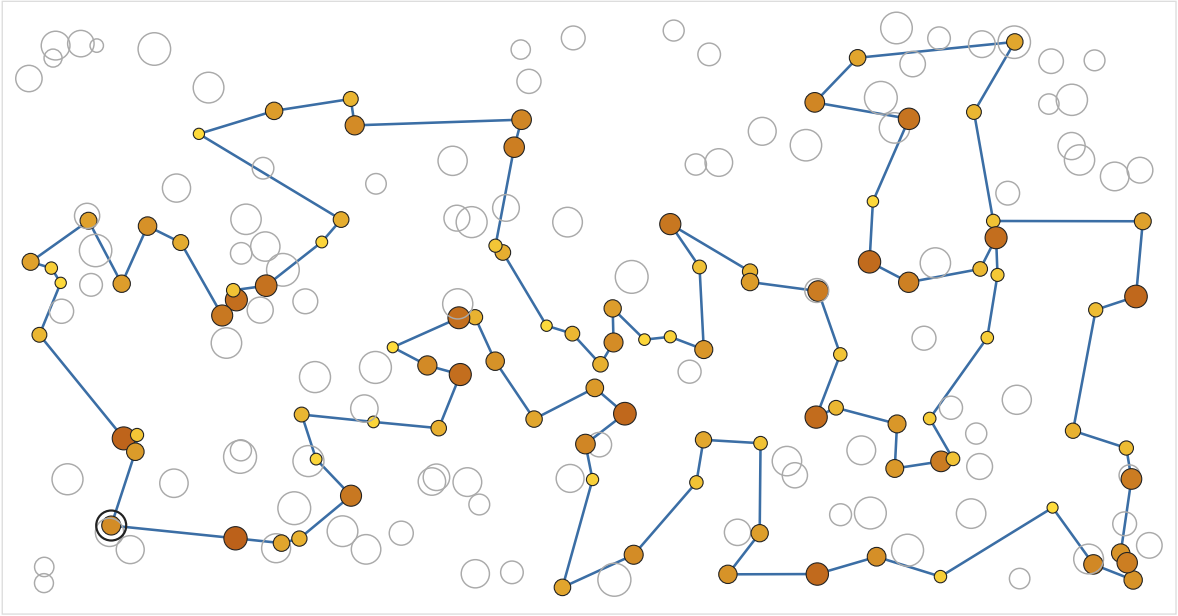
Method	TSPA valid	TSPA local optima	TSPB valid	TSPB local optima
Steepest LS, baseline (all moves evaluated every iteration)	**200 / 200**	**200 / 200**	**200 / 200**	**200 / 200**
Steepest LS, list of improving moves (LM)	**200 / 200**	**200 / 200**	**200 / 200**	**200 / 200**

All runs valid and all runs are local optima.

Instance TSPA

Steepest LS, baseline (all moves evaluated every iteration)

Best of 200 valid runs: run #33 with objective **73856**.



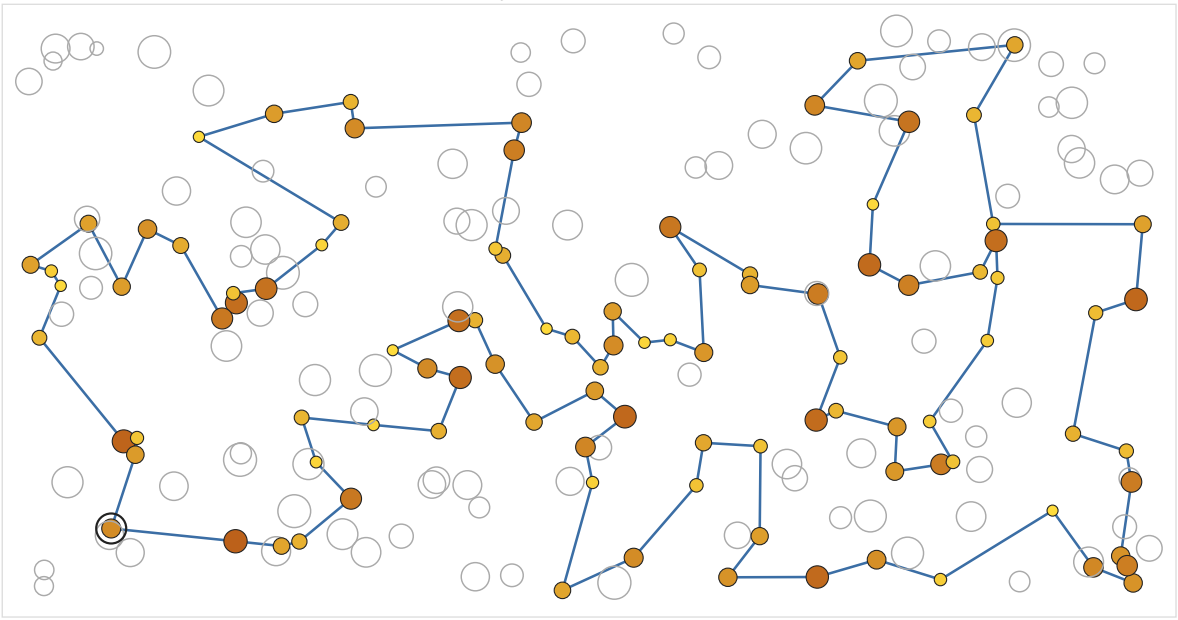
node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
69, 92, 171, 2, 189, 187, 30, 141, 87, 102, 28, 180, 156, 35, 185, 45, 140, 41, 73, 135, 114, 181, 58, 84, 134,
158, 192, 111, 182, 57, 107, 88, 78, 193, 153, 170, 159, 130, 47, 37, 147, 1, 178, 29, 136, 186, 184, 164, 142,
85, 3, 169, 50, 93, 106, 89, 42, 151, 150, 176, 72, 174, 8, 54, 9, 143, 33, 131, 36, 177, 52, 179, 53, 144, 67,
16, 51, 139, 112, 71, 157, 91, 80, 76, 132, 90, 123, 83, 160, 152, 108, 49, 60, 116, 149, 63, 168, 95, 97, 127
```

Steepest LS, list of improving moves (LM)

Best of 200 valid runs: run #33 with objective **73856**.



node cost 0 → 1997 (colour and size) · hollow grey = not selected · ring = first node

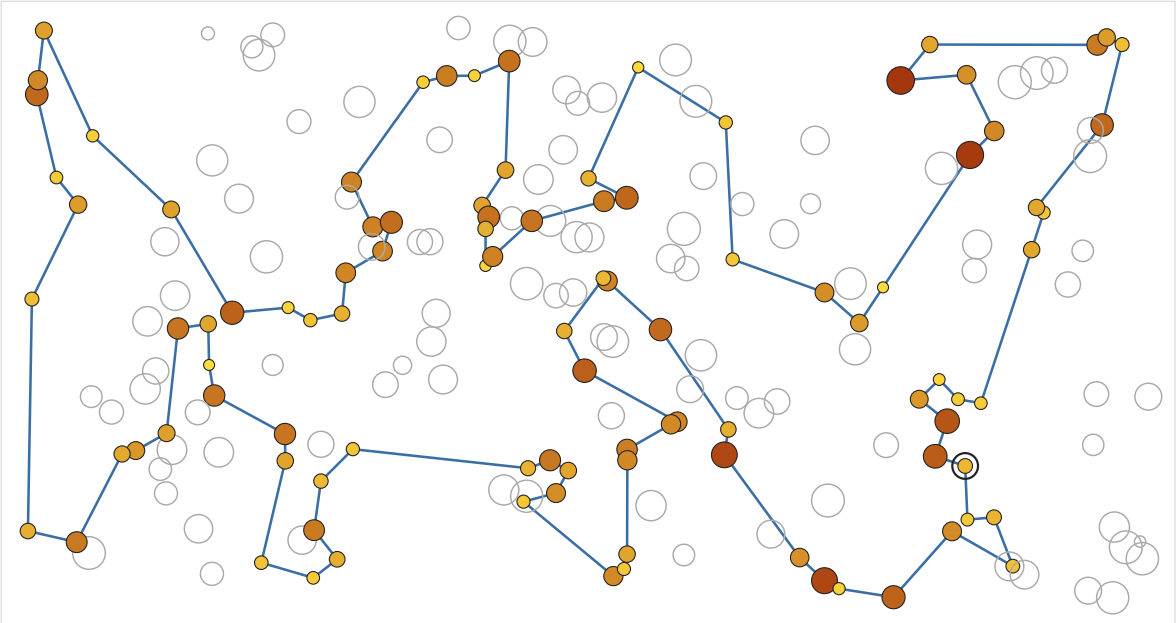
Best solution (node indices, from 0)

```
69, 92, 171, 2, 189, 187, 30, 141, 87, 102, 28, 180, 156, 35, 185, 45, 140, 41, 73, 135, 114, 181, 58, 84, 134,
158, 192, 111, 182, 57, 107, 88, 78, 193, 153, 170, 159, 130, 47, 37, 147, 1, 178, 29, 136, 186, 184, 164, 142,
85, 3, 169, 50, 93, 106, 89, 42, 151, 150, 176, 72, 174, 8, 54, 9, 143, 33, 131, 36, 177, 52, 179, 53, 144, 67,
16, 51, 139, 112, 71, 157, 91, 80, 76, 132, 90, 123, 83, 160, 152, 108, 49, 60, 116, 149, 63, 168, 95, 97, 127
```

Instance TSPB

Steepest LS, baseline (all moves evaluated every iteration)

Best of 200 valid runs: run #185 with objective **50402**.



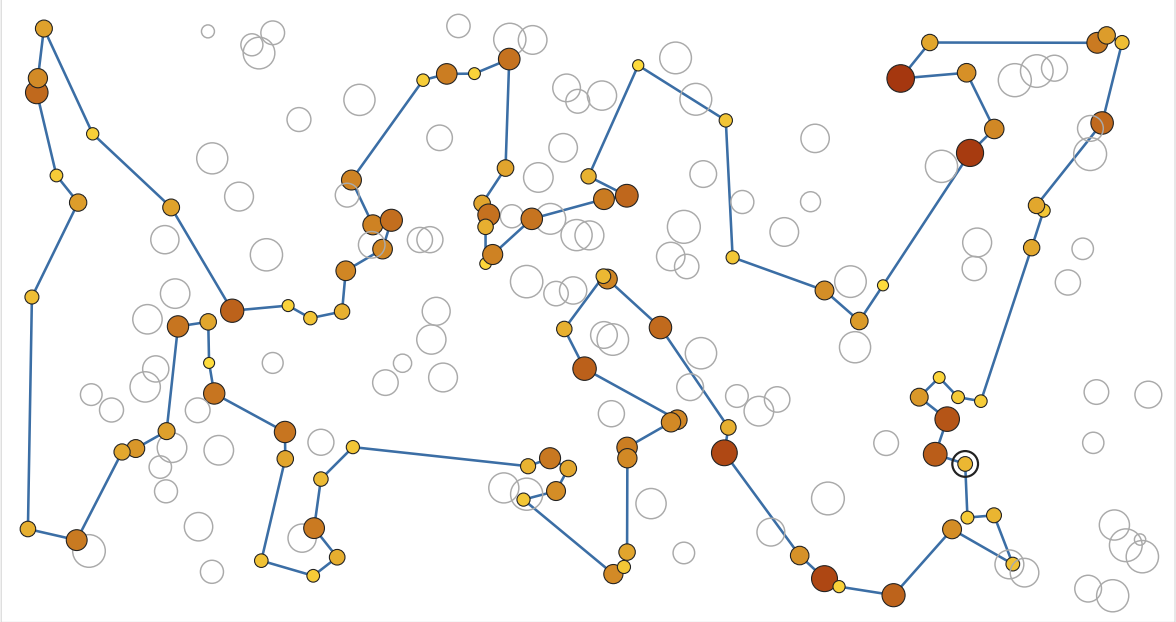
node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
153, 182, 52, 80, 127, 129, 90, 10, 114, 196, 61, 105, 94, 81, 190, 194, 70, 187, 18, 54, 192, 162, 76, 57, 89,
72, 86, 46, 117, 160, 69, 172, 136, 133, 120, 48, 96, 198, 121, 130, 30, 183, 21, 38, 47, 140, 154, 26, 158,
149, 37, 146, 13, 28, 145, 65, 1, 174, 132, 29, 6, 177, 125, 170, 115, 155, 71, 175, 134, 45, 157, 60, 40, 59,
73, 167, 51, 85, 82, 87, 179, 181, 152, 34, 24, 98, 151, 184, 3, 27, 67, 31, 99, 39, 171, 17, 74, 2, 14, 20
```

Steepest LS, list of improving moves (LM)

Best of 200 valid runs: run #185 with objective **50402**.



node cost 3 → 999 (colour and size) · hollow grey = not selected · ring = first node

Best solution (node indices, from 0)

```
153, 182, 52, 80, 127, 129, 90, 10, 114, 196, 61, 105, 94, 81, 190, 194, 70, 187, 18, 54, 192, 162, 76, 57, 89,
72, 86, 46, 117, 160, 69, 172, 136, 133, 120, 48, 96, 198, 121, 130, 30, 183, 21, 38, 47, 140, 154, 26, 158,
149, 37, 146, 13, 28, 145, 65, 1, 174, 132, 29, 6, 177, 125, 170, 115, 155, 71, 175, 134, 45, 157, 60, 40, 59,
73, 167, 51, 85, 82, 87, 179, 181, 152, 34, 24, 98, 151, 184, 3, 27, 67, 31, 99, 39, 171, 17, 74, 2, 14, 20
```

Source code: https://github.com/CodeSencor/evolutionary-computation

Conclusions

The list of improving moves does not change what the search does, only how fast it finds the next move: LM always holds every improving move of the current solution, so the best applicable move in LM is the move the baseline would pick. The quality of both methods is therefore the same (the small differences in the averages come only from ties between moves with equal Δ), and every run of both methods ends in a local optimum. LM is about 2.5 times as fast. After a move only the moves touching the few changed edges and nodes are evaluated, instead of the whole neighbourhood of about 15 000 moves, but this gain is partly eaten by the initial evaluation (which includes the relatively inverted moves) and by keeping the sorted list: from a random start a large part of the evaluated moves is improving and has to be inserted into LM. The fallback full evaluation (step 7) never found a new move, which confirms that the incremental update misses nothing.